

DEMONSTRATING FTP VS SFTP

SECURITY

Introduction

Organizations transfer files daily across networks, including documents, databases, backups, and confidential records.

Two common protocols used for file transfer are:

- FTP (File Transfer Protocol)
- SFTP (Secure File Transfer Protocol)

Although both perform the same function, they differ significantly in security.

This project demonstrates how FTP and SFTP operate and compares their security using packet analysis with Wireshark.

Objectives

- Demonstrate FTP and SFTP Security
- Wireshark packet capture analysis for FTP and SFTP
- Comparison of FTP and SFTP security
- Discuss the risk associated with FTP
- Explain why SFTP should be used and recommended

Tools Used:

Tools	Purpose
FileZilla Server	For FTP Configuration
Kali Linux	FTP Client Machine/SFTP Demonstration
OpenSSH	SFTP Service Running
Wireshark	FTP and SFTP Packet Capture Analysis

File Transfer Protocol (FTP) and Secure File Transfer Protocol (SFTP) are usually used

methods for transferring files across networks. FTP provides file transfer functionality but does not encrypt broadcast data, making it at risk to sniffing.

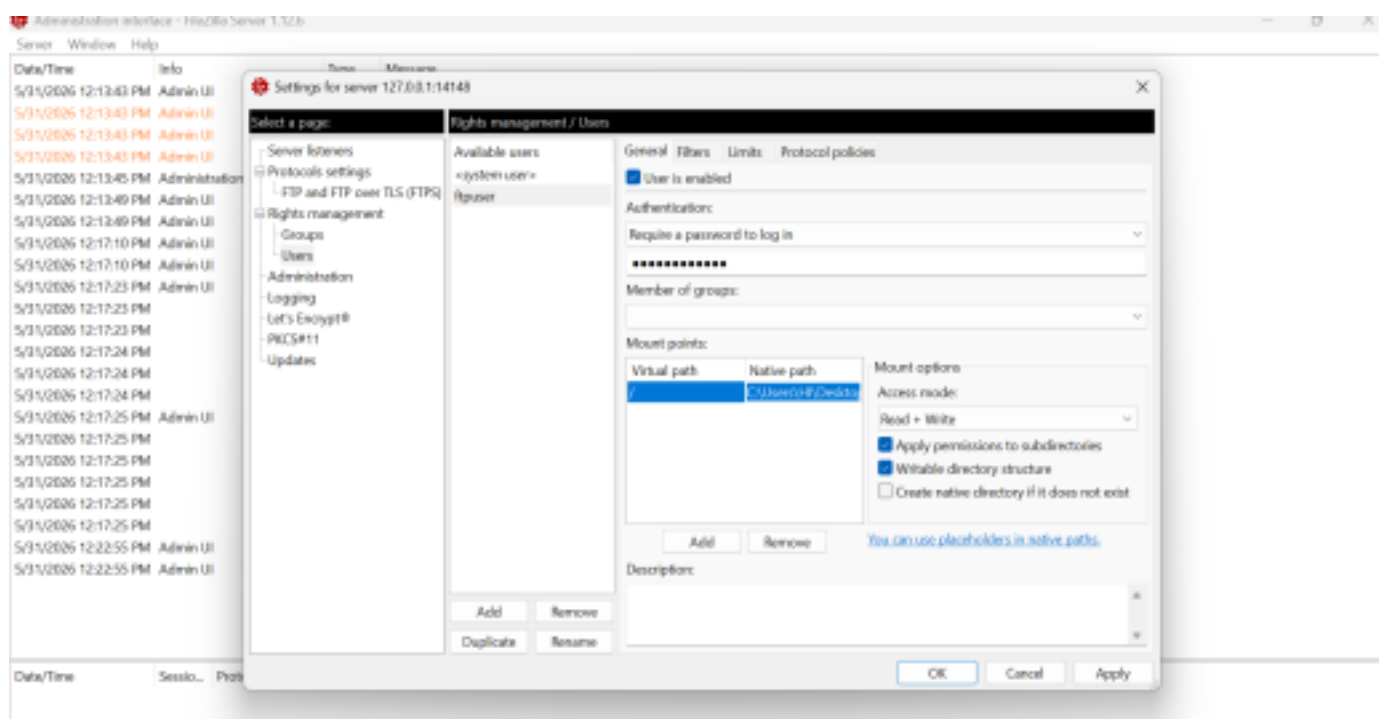
SFTP, which operates over the Secure Shell (SSH) protocol, encrypts all information transfer between the client and server, ensuring confidentiality and integrity.

This project shows the differences between FTP and SFTP security by capturing and analyzing network traffic using Wireshark. This project shows how FTP exposes confidential information while SFTP protects it through encryption.

1. FTP ENVIRONMENT SETUP

A local FTP environment was set up using FileZilla Server. An FTP user account (**ftpuser**) was created with a password (**ftpuser12345**). Also a shared folder was configured to allow file transfers between the FTP server and client.

A file named **myproject4.txt.txt** was set up and used during the file transfer activity.



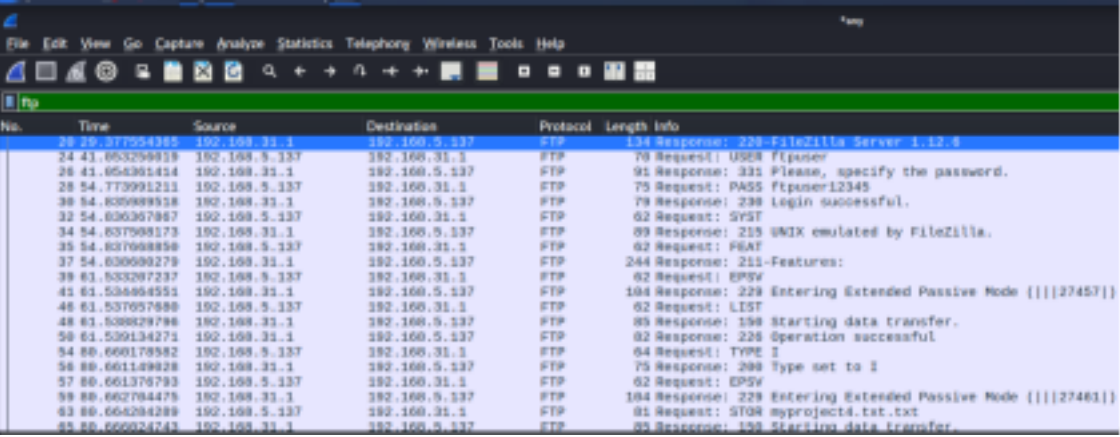
2. SUCCESSFUL FTP LOGIN

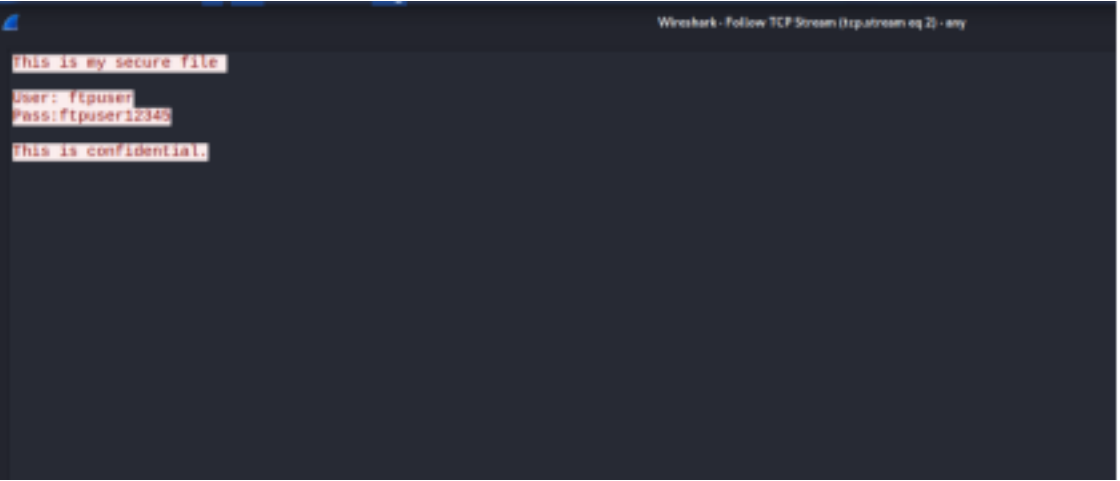
Kali Linux was used as the FTP client to link to the FileZilla Server running on Windows through the server's IP address. The Login was Successful, allowing file transfer operations.

(myproject4.txt.txt) was uploaded successfully through the FTP connection.

3. FTP TRAFFIC CAPTURE AND ANALYSIS

Wireshark was used to capture network packets generated during the FTP access period. The captured traffic was evaluated to determine whether confidential details could be viewed by an attacker tracking the network.

Wireshark Showing USER Command	 The image shows a Wireshark packet capture of an FTP session. The packet list table includes columns for No., Time, Source, Destination, Protocol, Length, and Info. The session starts with a 194 Response from 192.168.5.137 to 192.108.31.1. Subsequent packets show a 70 Request for USER ftpuser, a 91 Response asking for a password, a 75 Request for PASS ftpuser12345, a 79 Response confirming login success, a 62 Request for SYST, an 89 Response indicating UNIX emulation, a 62 Request for PSTAT, a 244 Response with features, a 62 Request for EPSV, a 104 Response entering extended passive mode, a 62 Request for LIST, an 85 Response starting data transfer, a 62 Request for 226, a 64 Request for TYPE I, a 75 Response setting type to I, a 62 Request for EPSV, a 104 Response entering extended passive mode, a 61 Request for STOR myproject4.txt.txt, and an 85 Response starting data transfer.
Wireshark Showing File Data Transfer	 The image shows the packet details pane in Wireshark for a selected packet. It displays the FileZilla protocol structure, including the 229 Entering Extended Passive Mode command, the 150 Starting data transfer command, the 226 Operation successful command, the 1571 I command, the 150 Type set to I command, the 229 Entering Extended Passive Mode command, the 0104 myproject4.txt.txt command, the 150 Starting data transfer command, the 226 Operation successful command, the 041 command, and the 223 Goodbye command. The packet list at the bottom shows the packet is from 192.168.5.137 to 192.108.31.1.

Wireshark Showing Readable File Content	 The image shows the packet details pane in Wireshark for a selected packet. It displays the FileZilla protocol structure, including the 229 Entering Extended Passive Mode command, the 150 Starting data transfer command, the 226 Operation successful command, the 1571 I command, the 150 Type set to I command, the 229 Entering Extended Passive Mode command, the 0104 myproject4.txt.txt command, the 150 Starting data transfer command, the 226 Operation successful command, the 041 command, and the 223 Goodbye command. The packet list at the bottom shows the packet is from 192.168.5.137 to 192.108.31.1.
---	---

During the FTP interaction, Wireshark was used to record and analyze network traffic between the Kali Linux FTP client and the FileZilla FTP server. The analysis indicated that FTP transfers all data exchange in plaintext without encryption.

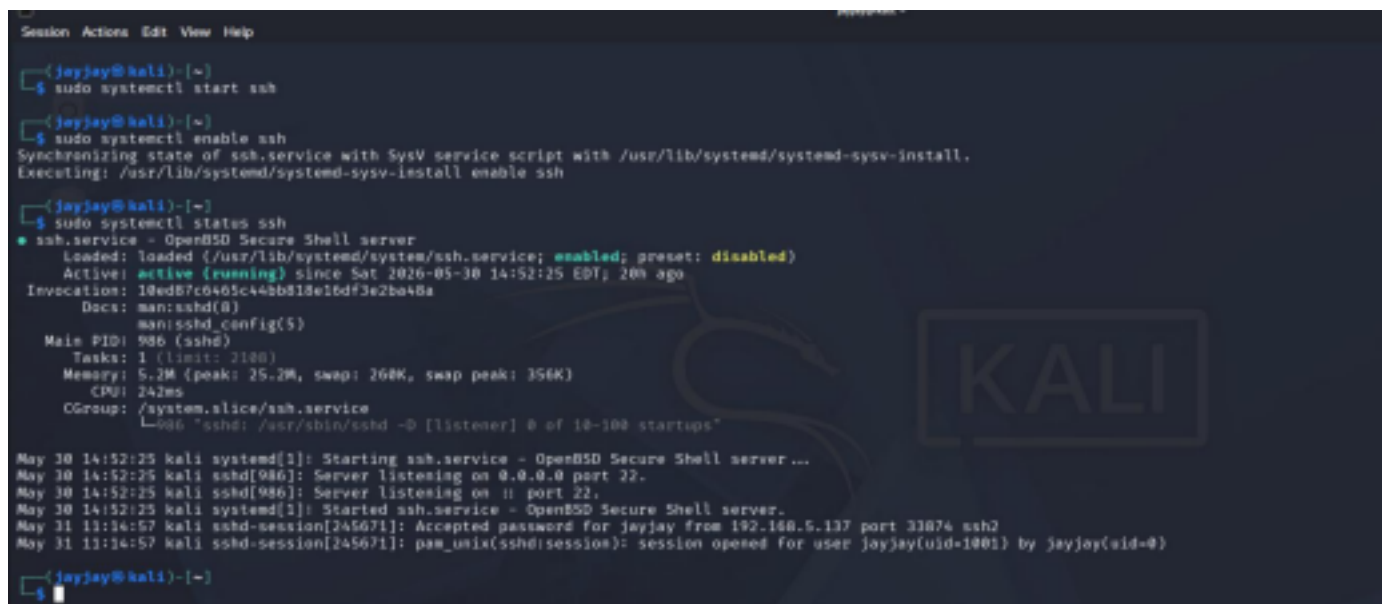
The Captured packets showed the Username and Password used for authentication through the FTP USER and PASS commands. The records of the sent file (**myproject4.txt.txt**) was visible and readable within the packet capture. This shows that any attacker monitoring the network could intercept confidential data, including login details and transferred information.

These outcomes confirm that FTP does not offer confidentiality or protection against sniffing attacks, making it an unprotected protocol for transferring sensitive information.

SFTP ENVIRONMENT SETUP

To provide secure file transfer, the OpenSSH service accessible on Kali Linux was used to set up an SFTP connection. A sample file (**sftp_groupproject4.txt**) was created specifically for the SFTP demonstration and transferred through the encrypted connection.

1. OpenSSH Service Running

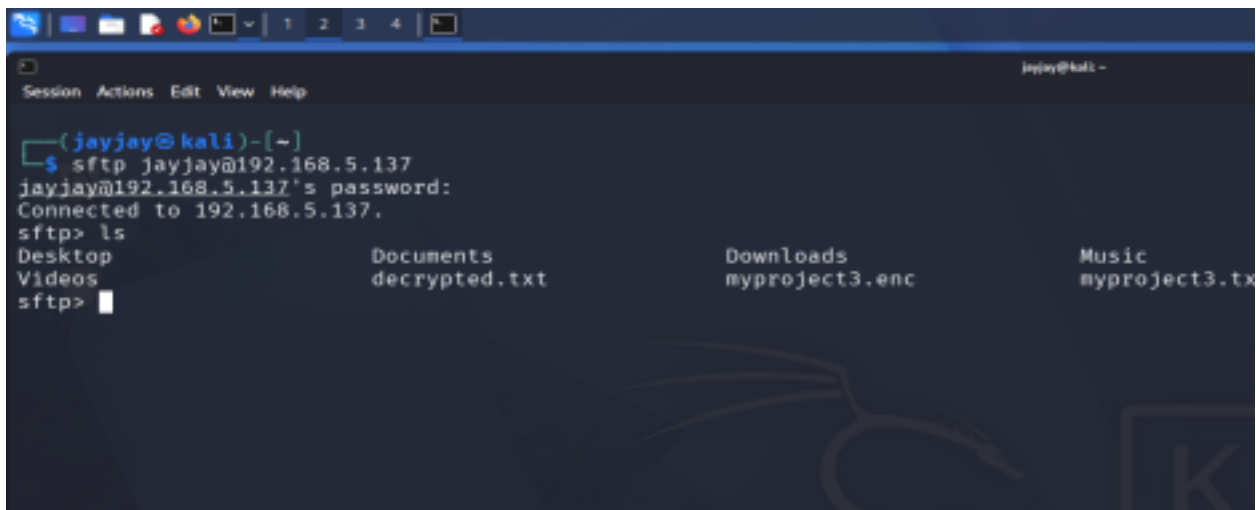
A terminal window on Kali Linux showing the steps to start and enable the OpenSSH service. The user 'jayjay' runs 'sudo systemctl start ssh' and 'sudo systemctl enable ssh'. Then, 'sudo systemctl status ssh' is run, showing the service is active and running. The status output includes details like 'Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)', 'Active: active (running) since Sat 2026-05-30 14:52:25 EDT; 20s ago', and 'Main PID: 986 (sshd)'. Below this, a log shows the service starting and listening on port 22. Finally, a log entry shows a successful login for 'jayjay' from IP 192.168.5.137 on port 33874.

```
Session Actions Edit View Help
(jayjay@kali)~$ sudo systemctl start ssh
(jayjay@kali)~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
(jayjay@kali)~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-05-30 14:52:25 EDT; 20s ago
     Invocation: 10ed87c6465c446b318e16df3e2ba48a
       Docs: man:sshd(8)
            man:sshd_config(5)
   Main PID: 986 (sshd)
     Tasks: 1 (limit: 2100)
    Memory: 5.2M (peak: 25.2M, swap: 268K, swap peak: 356K)
       CPU: 242ms
   CGroup: /system.slice/ssh.service
           └─sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

May 30 14:52:25 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
May 30 14:52:25 kali sshd[986]: Server listening on 0.0.0.0 port 22.
May 30 14:52:25 kali sshd[986]: Server listening on :: port 22.
May 30 14:52:25 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
May 31 11:14:57 kali sshd-session[245671]: Accepted password for jayjay from 192.168.5.137 port 33874 ssh2
May 31 11:14:57 kali sshd-session[245671]: pam_unix(sshd:session): session opened for user jayjay(uid=1001) by jayjay(uid=0)
(jayjay@kali)~$
```

2. SUCCESSFUL SFTP LOGIN

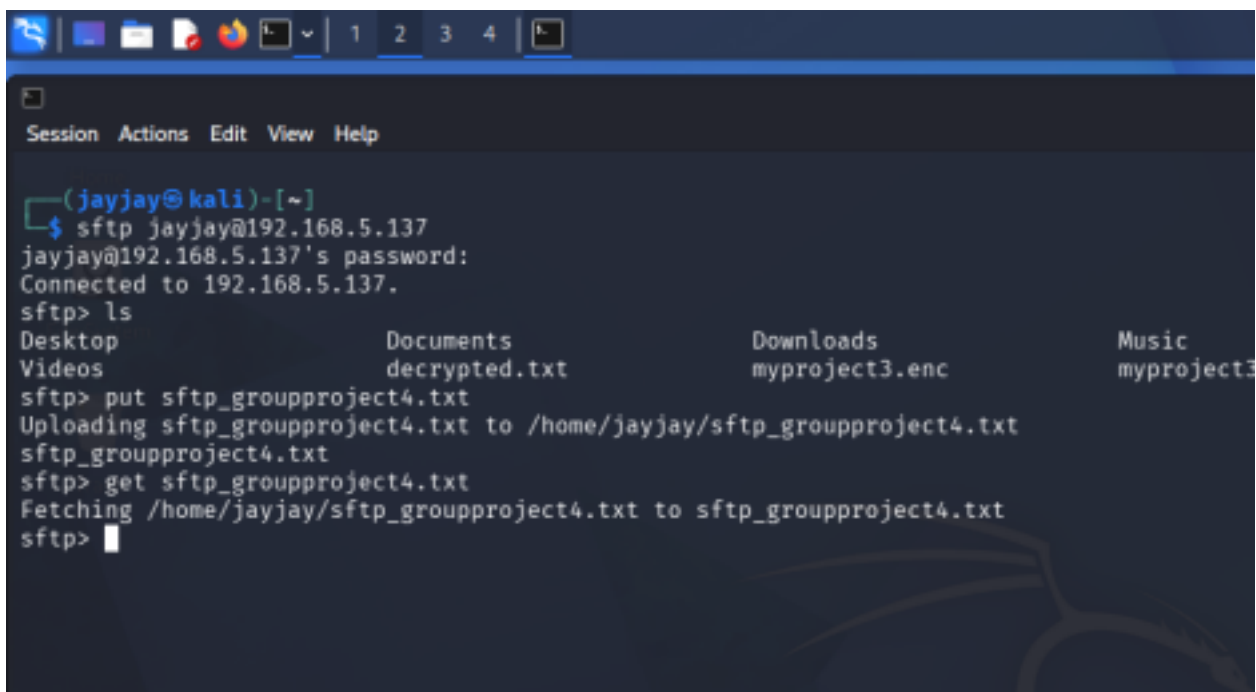
The SFTP client successfully connected to the OpenSSH server running on kali Linux using the server's IP address and valid Login details.



```
(jayjay@kali)-[~]
$ sftp jayjay@192.168.5.137
jayjay@192.168.5.137's password:
Connected to 192.168.5.137.
sftp> ls
Desktop          Documents        Downloads        Music
Videos           decrypted.txt    myproject3.enc  myproject3.tx
sftp>
```

3. SFTP FILE TRANSFER

The SFTP text file (**sftp_groupproject4.txt**) was accurately sent through the encrypted SFTP connection



```
(jayjay@kali)-[~]
$ sftp jayjay@192.168.5.137
jayjay@192.168.5.137's password:
Connected to 192.168.5.137.
sftp> ls
Desktop          Documents        Downloads        Music
Videos           decrypted.txt    myproject3.enc  myproject3
sftp> put sftp_groupproject4.txt
Uploading sftp_groupproject4.txt to /home/jayjay/sftp_groupproject4.txt
sftp_groupproject4.txt
sftp> get sftp_groupproject4.txt
Fetching /home/jayjay/sftp_groupproject4.txt to sftp_groupproject4.txt
sftp>
```

4. CAPTURE SSH TRAFFIC

Wireshark was used to capture network traffic generated during the SFTP connection. The packet capture was analyzed to establish whether credentials and file contents could be viewed.

No.	Time	Source	Destination	Protocol	Length	Info
30	94.302569058	192.168.5.137	192.168.5.137	SSHv2	160	Client: Protocol (SSH-2.0-OpenSSH_18.0p1 Debian-3)
31	94.304518265	192.168.5.137	192.168.5.137	SSHv2	160	Server: Protocol (SSH-2.0-OpenSSH_18.0p1 Debian-3)
32	94.306709338	192.168.5.137	192.168.5.137	SSHv2	1168	Server: Key Exchange Init
33	94.308927775	192.168.5.137	192.168.5.137	SSHv2	1748	Client: Key Exchange Init
34	94.329898888	192.168.5.137	192.168.5.137	SSHv2	3388	Client: PQ/T Hybrid Key Exchange Init
35	94.333954539	192.168.5.137	192.168.5.137	SSHv2	1712	Server: PQ/T Hybrid Key Exchange Reply, New Keys, Encrypted packet (len=348)
36	94.337298547	192.168.5.137	192.168.5.137	SSHv2	152	Client: New Keys, Encrypted packet (len=88)
41	94.368973981	192.168.5.137	192.168.5.137	SSHv2	112	Client: Encrypted packet (len=44)
43	94.368365078	192.168.5.137	192.168.5.137	SSHv2	112	Server: Encrypted packet (len=44)
45	94.388514428	192.168.5.137	192.168.5.137	SSHv2	138	Client: Encrypted packet (len=88)
46	94.393873074	192.168.5.137	192.168.5.137	SSHv2	368	Server: Encrypted packet (len=312)
47	95.074928670	192.168.5.137	192.168.5.137	SSHv2	218	Client: Encrypted packet (len=548)
48	95.724288223	192.168.5.137	192.168.5.137	SSHv2	88	Server: Encrypted packet (len=28)
51	95.724462032	192.168.5.137	192.168.5.137	SSHv2	188	Client: Encrypted packet (len=112)
53	95.729745889	192.168.5.137	192.168.5.137	SSHv2	698	Server: Encrypted packet (len=628)
54	95.72138917	192.168.5.137	192.168.5.137	SSHv2	112	Server: Encrypted packet (len=44)
55	95.72548281	192.168.5.137	192.168.5.137	SSHv2	258	Client: Encrypted packet (len=588)
58	95.72174725	192.168.5.137	192.168.5.137	SSHv2	148	Server: Encrypted packet (len=72)
59	95.72338282	192.168.5.137	192.168.5.137	SSHv2	112	Client: Encrypted packet (len=44)
63	95.71935032	192.168.5.137	192.168.5.137	SSHv2	434	Server: Encrypted packet (len=364)

5. ENCRYPTED SFTP TRAFFIC

Unlike FTP traffic, the packet information appeared encrypted and unreadable. User credentials and file contents could not be viewed within the captured packets.

72	95.621357923	192.168.5.137	192.168.5.137	SSHv2	138	Server: Encrypted packet (len=88)
73	95.621859659	192.168.5.137	192.168.5.137	SSHv2	128	Client: Encrypted packet (len=52)
74	95.622348743	192.168.5.137	192.168.5.137	SSHv2	138	Server: Encrypted packet (len=88)
75	95.622788835	192.168.5.137	192.168.5.137	SSHv2	178	Client: Encrypted packet (len=108)

0100	 = Version: 4	0000	88 00 03 04 06 06 00 00	00 00 00 00 00 00 88 00	
.... 0101	= Header Length: 20 bytes (5)	0010	45 00 00 c8 07 38 40 00	40 06 46 3d c0 a8 85 89	hg @ @ F.....
+ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)		0020	c8 a8 05 05 c6 0e 00 16	16 b6 77 37 ab 96 96 8a	
Total Length: 104		0030	88 18 01 2f 8c bd 99 00	01 01 88 9a 70 f5 76 8d	9.....x v.....
Identification: 0x079e (28512)		0040	78 f5 76 0d 1c 87 e7 03	79 43 29 f1 b6 ee 55 8e	x v 03 -C)---U.....
+ 010... = Flags: 0x2, Don't Fragment		0050	52 90 38 0e f0 3c 62 53	09 55 4d c9 0f a5 5f 83	R 0 <ES -M+..._C.....
...0 0000 0000 0000 = Fragment Offset: 0		0060	c5 e1 20 24 f8 fc 0f d0	c5 f6 83 1d 0f 46 f6 7f	...S...-...qF.....
Time to Live: 64		0070	e4 b5 79 52 52 ab 95 20		...ySR--{.....
Protocol: TCP [6]					
Header Checksum: 0x468d [validation disabled]					
[Header checksum status: Unverified]					
Source Address: 192.168.5.137					
Destination Address: 192.168.5.137					
[Stream index: 1]					
+ Transmission Control Protocol, Src Port: 56782, Dst Port: 22, Seq: 3006, Ack: 4414, Len: 52					
+ SSH Protocol					
+ SSH Version 2 (encryption:chacha20-poly1305openssh.com mac:<implicit> compression:none)					
Packet Length (encrypted): 404fe793					
Encrypted Packet: f34329f1bbe559e520c300efb3c625309504d0bfaf5f935e12024f8fc8f80					
MAC: e3fcd031d6f48f57febd795252ab9620					
[Direction: Client to Server]					

COMPARATIVE ANALYSIS

The FTP and SFTP packet captures were compared to evaluate their security characteristics

FTP PACKET

No.	Time	Source	Destination	Protocol	Length	Info
28	29.377554365	192.168.31.1	192.168.5.137	FTP	134	Response: 228-FileZilla Server 1.12.6
24	41.853256019	192.168.5.137	192.168.31.1	FTP	70	Request: USER ftpuser
26	41.854361414	192.168.31.1	192.168.5.137	FTP	91	Response: 331 Please, specify the password.
28	54.773991211	192.168.5.137	192.168.31.1	FTP	75	Request: PASS ftpuser12345
30	54.835989518	192.168.31.1	192.168.5.137	FTP	79	Response: 230 Login successful.
32	54.836367867	192.168.5.137	192.168.31.1	FTP	62	Request: SYST
34	54.837506173	192.168.31.1	192.168.5.137	FTP	89	Response: 215 UNIX emulated by FileZilla.
35	54.837668850	192.168.5.137	192.168.31.1	FTP	62	Request: FEAT
37	54.838680279	192.168.31.1	192.168.5.137	FTP	244	Response: 211-Features:
39	61.533207237	192.168.5.137	192.168.31.1	FTP	62	Request: EPSV
41	61.534464551	192.168.31.1	192.168.5.137	FTP	184	Response: 229 Entering Extended Passive Mode (27457)
46	61.537657680	192.168.5.137	192.168.31.1	FTP	62	Request: LIST
48	61.538829796	192.168.31.1	192.168.5.137	FTP	85	Response: 150 Starting data transfer.
50	61.539134271	192.168.31.1	192.168.5.137	FTP	82	Response: 226 Operation successful
54	80.660178582	192.168.5.137	192.168.31.1	FTP	64	Request: TYPE I
56	80.661149028	192.168.31.1	192.168.5.137	FTP	75	Response: 200 Type set to I
57	80.661376793	192.168.5.137	192.168.31.1	FTP	62	Request: EPSV
59	80.662704475	192.168.31.1	192.168.5.137	FTP	184	Response: 229 Entering Extended Passive Mode (27461)
63	80.664284289	192.168.5.137	192.168.31.1	FTP	81	Request: STOR myproject4.txt.txt
65	80.666824743	192.168.31.1	192.168.5.137	FTP	85	Response: 150 Starting data transfer.

SFTP PACKET

No.	Time	Source	Destination	Protocol	Length	Info
30	94.283589956	192.168.5.137	192.168.5.137	SSHv2	501	Client: Protocol (SSH-2.0-OpenSSH_6.3p1 Debian-1)
32	94.286798328	192.168.5.137	192.168.5.137	SSHv2	501	Server: Protocol (SSH-2.0-OpenSSH_6.3p1 Debian-1)
34	94.286798328	192.168.5.137	192.168.5.137	SSHv2	1508	Server: Key Exchange Init
36	94.328068856	192.168.5.137	192.168.5.137	SSHv2	1748	Client: Key Exchange Init
38	94.333054538	192.168.5.137	192.168.5.137	SSHv2	1360	Client: PQ/T Hybrid Key Exchange Init
39	94.337209447	192.168.5.137	192.168.5.137	SSHv2	1712	Server: PQ/T Hybrid Key Exchange Reply, New Keys, Encrypted packet (len=338)
41	94.337209447	192.168.5.137	192.168.5.137	SSHv2	152	Client: New Keys, Encrypted packet (len=88)
43	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Client: Encrypted packet (len=82)
45	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Server: Encrypted packet (len=44)
47	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Client: Encrypted packet (len=86)
49	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Server: Encrypted packet (len=312)
51	94.380612981	192.168.5.137	192.168.5.137	SSHv2	216	Client: Encrypted packet (len=148)
53	94.380612981	192.168.5.137	192.168.5.137	SSHv2	96	Server: Encrypted packet (len=28)
55	94.380612981	192.168.5.137	192.168.5.137	SSHv2	500	Client: Encrypted packet (len=112)
57	94.380612981	192.168.5.137	192.168.5.137	SSHv2	896	Server: Encrypted packet (len=528)
59	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Client: Encrypted packet (len=44)
61	94.380612981	192.168.5.137	192.168.5.137	SSHv2	500	Server: Encrypted packet (len=188)
63	94.380612981	192.168.5.137	192.168.5.137	SSHv2	540	Client: Encrypted packet (len=72)
65	94.380612981	192.168.5.137	192.168.5.137	SSHv2	512	Server: Encrypted packet (len=82)
67	94.380612981	192.168.5.137	192.168.5.137	SSHv2	424	Client: Encrypted packet (len=368)

COMPARISON OF FTP AND SFTP TRAFFIC

FTP	SFTP
Provides zero encryption. All commands and data pass the network interface in raw, readable plaintext.	All packets are encrypted, data and commands are all encrypted before leaving the system.
Wireshark FTP Capture: it showed commands and contexts of the file like USER (ftpuser) and PASS (ftpuser12345) in plaintext.	Shows the protocols SSHv2 and key exchange handshakes then all subsequent data blocks are all encrypted.
FTP Risks: Highly vulnerable to man-in-the middle interception(MITM), data harvesting and packet sniffing. Hackers monitoring local network interfaces can capture data and	SFTP Protection: Neutralizes sniffing threats via SSH encryption, reduces man-in-the middle attacks during sessions. It also adds integrity to ensure packets are not tampered with

credentials.	or dropped during transit.
--------------	----------------------------

SECURITY RISKS OF FTP

FTP presents several security risks because it does not offer encryption for data transferred across the network. user, pass commands, and are sent in readable data, making them detectable to attackers who record the network traffic.

On this project, Wireshark analysis displayed that FTP credentials and transmitted file records could be seen from the captured packets. This shows how attackers can disrupt confidential information through packet sniffing methods.

FTP also lacks internal methods to ensure data confidential and integrity. As a result, confidential information may be revealed, changed, or connected to by unauthorized individuals during dissemination.

These vulnerabilities make FTP unacceptable for transmitting sensitive information such as financial records, employee data, customer information, and business documents. For this reason, safeguarded choices such as SFTP are recommended because they protect data through encryption and protected verification.

The packet capture illustrates that FTP transfer authentication credentials and file contents without encryption. This makes FTP exposed to packet sniffing attacks, credential theft, and unapproved access.

HOW SFTP PROVIDES CONFIDENTIALITY AND INTEGRITY

Confidentiality:

Ensures that unauthorized users and attackers won't be able to access or read the data being transferred with the help of SFTP.

The encrypted SSH Protocol (SSHv2): Before data is transferred across the network, SFTP uses the SSH Protocol to build a secure route between the client and the server.

SFTP uses a two-step locking process to keep data hidden. It uses asymmetric encryption at the very beginning (the handshakes) to safely pass a temporary symmetric key to the server then switches back to symmetric encryption which is faster.

Integrity (Anti-Tampering):

Integrity ensures that data is not altered, tampered or modified during transit.

Creating a Digital fingerprint (or Checksum): SFTP uses digital fingerprint (hashes) so hackers or unauthorized users can't alter or modify a file without getting caught. If hackers or unauthorized users tamper with the file, the fingerprint will change which tells the receiver or owner that the file has been altered. The receiver reassesses the hash to check if the hash is safe and unaltered.

REAL-WORLD IMPLICATIONS

Banks and organizations constantly transfer payrolls files, payment instructions, operation records, and financial reports with corporate client and business partners, because these files contain confidential financial data, the banking system widely uses SFTP to protect data during communication.

Why FTP Is a Risk?

- FTP transfer usernames, passwords, and information in readable data
- Attackers observing network traffic can possibly divert logins and confidential files.
- Revealed financial data can lead to scam, regulatory penalties, and reputational damage

Why SFTP Is Preferred?

- SFTP encrypts authentication credentials
- SFTP encrypts all transmitted files using SSH
- Captured network traffic is unreadable without the encryption keys
- It helps agencies meet security and fulfilment requirements.

Implications:

- Safeguarding of customer and employee monetary information
- Lessened risk of data breaches and credential fraud
- Adherence with industry security standards
- Secure communication between companies and banking partners

Enterprises regularly transmit confidential records across networks. Using FTP could leak classified information to exploiters observing network data flow. SFTP is recommended because it encrypts communications, protects user credentials, and ensures the confidentiality and integrity of transferred files.

ENCRYPTION MECHANISM BEHIND SFTP (SSH TUNNEL)

In this project, SFTP was used to transfer files securely between the client (Kali Linux) and the SSH server. Unlike FTP, where data is seen in plaintext.

When the client (Kali Linux) initiated an SFTP connection, an SSH handshake occurred between the client and the server. During the process, encryption keys were exchanged and a secure encrypted tunnel was initiated. Once the tunnel is created, all communications between the client and the server is encrypted before being transmitted across the network in

case of man-in-the-middle sniffing.

The encryption protected usernames, passwords, file contents and commands from being intercepted by unauthorized users. As a result, when the network traffic capture tool (Wireshark) filtered the packets, the data being transferred was not in plaintext.

The packet capture showed encrypted SSH packets. This demonstrates that SFTP provides confidentiality by preventing unauthorized users from viewing sensitive information when monitoring traffic.

SFTP includes integrity verification mechanisms that ensure transferred data is not modified during transmission. Any unauthorized tampering of the data can be detected, helping to maintain the integrity of the files.

The results of the Wireshark analysis show that SFTP ensures protection of transferred data through the use of encrypted SSH tunnel, making it more secure than FTP.

IMPORTANCE OF USING SECURE PROTOCOLS IN EVERYDAY NETWORK COMMUNICATION

- **PROTECTING PRIVACY:** secure protocols encrypt data in transit, guarding sensitive information like (passwords, financial details, or personal messages) from interception by unauthorized personnel. Without encryption, our usual operations like logging into our social media accounts, sending emails or even shopping online might reveal personal data to malicious actors.
- **SAFEGUARDING TRANSACTIONS:** online banking, e-commerce, and digital payments all use secure protocols to guarantee the integrity and privacy of financial exchanges. TLS (Transport Layer Security) helps to ensure the integrity of the file which simply means that the data reach their destination securely and intact
- **PREVENTING CYBER ATTACKS:** Secure protocols helps to protect against attacks like traffic interception attack where cyber criminal's intercepts and alter the communication. They also help to identify authenticity ensuring the site is genuine and not a fraudulent duplicate

Conclusion

This project demonstrated that:

- FTP exposes usernames, passwords, and files in plaintext.
- Wireshark can easily capture FTP credentials.
- SFTP encrypts all communications through SSH.
- SFTP provides confidentiality, integrity, and stronger security.

Therefore, SFTP is the recommended protocol for secure file transfer in modern organizations.